Wednesday: 08:00-12:00
Class: D2416

Ders Adı	Kodu	Yılı	Yarıyılı	Süresi (T+U)	Kredisi	AKTS Kredisi
OBJECT ORIENTED PROGRAMMING	UYG203	2	3	4+0	4.0	6.0

Department: Software Development

Course Duration: 14 Weeks

Weekly Hours: 4 (2 hours Theory + 2 hours Laboratory)

Programming Language: Python 3.x

Prerequisites: Introduction to Programming

Course Description

This course introduces the principles of Object-Oriented Programming (OOP) using Python. Students learn how to design, implement, test, and maintain object-oriented software systems by applying modern software engineering principles. The course emphasizes object-oriented thinking, reusable software components, inheritance, polymorphism, abstraction, encapsulation, exception handling, file processing, and the development of medium-scale applications.

Learning Outcomes

Upon successful completion of this course, students will be able to:

1. Explain the principles of object-oriented programming.
2. Design software using classes and objects.
3. Implement reusable software components.
4. Apply encapsulation, inheritance, abstraction, and polymorphism.
5. Develop modular Python applications.
6. Read and write data using files.

7. Handle runtime exceptions effectively.
8. Build medium-scale object-oriented applications.
9. Apply software engineering best practices.
10. Document and test Python programs.

Week	Topic	Theory (2 Hours)	Laboratory (2 Hours)	Learning Outcomes
1	**Introduction to Object-Oriented Programming**	Course overview, Python review, Procedural vs. Object-Oriented Programming, OOP principles and benefits	Python environment setup (VS Code), first Python program, review exercises	Explain the differences between procedural and object-oriented programming and configure the Python development environment.
2	**Classes and Objects**	Classes, objects, constructors (`__init__()`), `self` , object lifecycle	Develop `Student` , `Book` , and `Product` classes	Create classes and instantiate objects using constructors.
3	**Attributes and Methods**	Instance variables, class variables, instance methods, object interaction	Develop `BankAccount` , `Employee` , and `Car` classes	Design classes containing attributes and behaviors.
4	**Encapsulation**	Information hiding, public/protected/private members, getters & setters, `@property`	Student Information System with encapsulated data	Apply encapsulation principles to protect object data.
5	**Inheritance**	Parent and child classes, code reuse, `super()` , inheritance hierarchy	Vehicle Management System (`Vehicle` , `Car` , `Motorcycle`)	Implement inheritance and reuse existing class functionality.
6	**Polymorphism**	Method overriding, dynamic binding, duck typing	Payment System (`CreditCard` , `PayPal` , `Cash`)	Apply polymorphism using overridden methods.
7	**Abstraction**	Abstract classes, abstract methods, Python `abc` module, interfaces	Shape hierarchy (`Rectangle` ,	Design abstract classes and implement

Week	Topic	Theory (2 Hours)	Laboratory (2 Hours)	Learning Outcomes
			`Circle`, `Triangle`)	concrete subclasses.
8	**Midterm Examination**	Written and practical examination	Practical coding assessment	Evaluate students' understanding of core OOP concepts.
9	**Composition and Aggregation**	"Has-a" relationships, object composition vs. inheritance, aggregation	University Management System (`Department`, `Student`, `Instructor`)	Model relationships between objects using composition and aggregation.
10	**Exception Handling**	Runtime errors, `try`, `except`, `finally`, `raise`, custom exceptions	ATM Simulation with exception handling	Build reliable programs using structured exception handling.
11	**File Handling**	Reading/writing text files, CSV, JSON, file organization	Student Grade Management System with persistent storage	Read and write structured data using Python file operations.
12	**Modules, Packages and Testing**	Modules, packages, imports, introduction to unit testing (`pytest`)	Create reusable utility package and write simple unit tests	Organize code into modules and validate functionality through testing.
13	**Object-Oriented Project Development**	Software architecture, UML class diagrams, project planning, documentation	Final project development (Library, Hospital, Hotel, Inventory System, etc.)	Apply OOP concepts to design and implement a medium-scale software project.
14	**Project Presentations & Course Review**	Code review, software engineering best practices, comprehensive review	Final project presentations and demonstrations	Present and evaluate object-oriented software solutions.